# État de l'art : Les systèmes de recommandation

Hiba Bederina

Août 2024

Les systèmes de recommandation ont été développés en réponse à l'expansion massive du contenu disponible en ligne. En effet, ils aident à réduire le temps et les efforts que les utilisateurs doivent consacrer pour trouver des informations pertinentes parmi une vaste gamme d'options sur Internet. Durant l'année 2024, environ 720 000 heures de vidéo sont téléchargées chaque jour sur YouTube, ce qui signifie qu'il faudrait des années à un utilisateur pour tout visionner (Sources: EarthWeb, DemandSage). De plus, Spotify compte environ 6 millions de podcasts, 350 000 livres audio, et 100 million de chansons disponibles (spotify). Quant à Amazon, la plateforme propose environ 600 millions de produits répertoriés sur son marketplace (amzscout).

En plus de chercher à améliorer la satisfaction des utilisateurs, les entreprises s'efforcent également de maximiser leur engagement ou leur alignement avec leur stratégies commerciale. Par conséquent, l'historique de navigation et toute interaction de l'utilisateur avec les plate-formes en ligne sont cruciaux pour le définir en tant que spectateur, auditeur, lecteur ou consommateur. Cela crée deux défis pour ces entreprises:

1. Comment les systèmes de recommandation équilibrent-ils la personnalisation et les préoccupations en matière de confidentialité ?

2. Quelles sont les implications éthiques de l'utilisation de systèmes de recommandation pour façonner le comportement des consommateurs ?

## 1 Procédure générale d'un système de recommandation

Les systèmes de recommandation se composent généralement de trois composants clés:

1. **Génération de candidats :** Ce composant génère un sous-ensemble réduit d'éléments à recommander parmi un vaste ensemble de milliers. Il limite les options à un nombre gérable pour un traitement ultérieur.

2. **Systèmes de notation :** Étant donné que la génération de candidats peut être réalisée par différentes méthodes, le système de notation stan-

dardise ces candidats en attribuant un score à chaque élément du sous-ensemble, en fonction de divers facteurs (géolocalisation des offres par exemple).

3. **Systèmes de reclassement "Ranking" :** Après la notation, le système applique des contraintes et des critères supplémentaires lié à l'objectif de la recommandation pour affiner la liste, produisant ainsi les recommandations finales classées qui sont présentées à l'utilisateur.

# 2 Les types des systèmes de recommandation

Les systèmes de recommandation peuvent être classés en fonction des informations utilisées pour prédire les préférences des utilisateurs. On distingue principalement quatre types : la recommandation non personnalisée, le filtrage collaboratif, les recommandations basées sur le contenu et les méthodes hybrides.

## 2.1 Recommandation non personnalisée

Les recommandations non personnalisées ne tiennent pas compte des préférences individuelles des utilisateurs ni du contexte spécifique. Elles se fondent plutôt sur des statistiques globales et des tendances générales observées dans l'ensemble de la population.

**Metriques:**

- La note moyenne attribuée à un élément spécifique par tous les utilisateurs.
- Le nombre total d'utilisateurs ayant évalué l'élément, indiquant sa popularité.
- Le pourcentage d'utilisateurs ayant attribué une note supérieure à un certain seuil, reflétant la proportion d'avis positifs.
- Le pourcentage d'utilisateurs ayant évalué à la fois l'élément A et l'élément B, mettant en lumière les éléments souvent appréciés ensemble (Ratio d'association/Co-occurrence).
- La corrélation Pearson entre les notes attribuées par le même utilisateur à deux éléments différents, révélant des modèles d'évaluation similaires et des affinités potentielles.
- La note moyenne d'un élément, segmentée par sexe, pour identifier les tendances et les différences dans les évaluations entre les sexes.
- Le pourcentage d'utilisateurs d'un sexe spécifique ayant attribué une note supérieure à un certain seuil, mettant en évidence les éléments les mieux notés pour chaque sexe.

De plus, les notes de base segmentées en fonction de facteurs tels que le sexe, la ville ou le statut économique sont des exemples de catégories facilement distinguées. Ces segments peuvent améliorer la qualité des recommandations lorsqu'on estime que des produits spécifiques sont mieux adaptés à chaque groupe.

- **Avantages :** Pas besoin de profils d'utilisateur ou d'historique de navigation pour déterminer les préférences ou prédire les futurs notes. Le système repose uniquement sur des statistiques simples.
- **Inconvénients :** Les statistiques fiables nécessitent un nombre substantiel d'interactions entre les utilisateurs et les articles. Il peut être difficile de fournir des recommandations nuancées, car les recommandations stéréotypées sont efficaces uniquement si des produits spécifiquement adaptés à chaque segment sont disponibles.

## 2.2 Filtrage collaboratif

Le filtrage collaboratif utilise le comportement collectif des utilisateurs pour améliorer les recommandations individuelles. Il se décline en trois variantes principales :

1. **Filtrage collaboratif utilisateur-utilisateur :** Cette approche recommande des articles à un utilisateur en fonction des préférences d'autres utilisateurs ayant des goûts similaires. Si deux utilisateurs ont aimé des articles semblables, le système suggérera à l'un des utilisateurs des articles appréciés par l'autre.

   Dans le filtrage collaboratif utilisateur-utilisateur, le processus suit les étapes suivantes dans un cadre de voisin le plus proche :

   (a) **Calcule de la similarité:** À partir de la matrice utilisateur-item, calculez la similarité entre tous les utilisateurs en utilisant la mesure de corrélation de Pearson.

   (b) **Identification des voisins:** Pour chaque utilisateur, identifiez les $K$ voisins les plus similaires en fonction des similarités calculées. Le nombre de voisins est généralement compris entre 20 et 30 (Herlocker et al., 2002).

   (c) **Prédiction des notes:** Prédisez les nouvelles notes pour l'utilisateur en utilisant les notes attribuées par ces $K$ voisins les plus proches. La formule de prédiction est :

   $$(\frac{\sum_{i=1}^{k} r_i w_i}{\sum_{i=1}^{k} w_i})$$

   où $r_i$ est la note donnée par le voisin $i$, et $w_i$ est le poids du voisin $i$. Cette formule calcule la note moyenne pondérée des voisins les plus proches.

Pour améliorer la précision, il est utile de considérer les écarts par rapport aux moyennes. Ainsi, la formule suivante ajuste la moyenne pondérée en tenant compte des écarts des voisins par rapport à leur propre moyenne :

$$(r_{user} + \frac{\sum_{i=1}^{k}(r_i - r_{neighborhood})w_i}{\sum_{i=1}^{k} w_i})$$

où $r_{user}$ est la note moyenne de l'utilisateur, et $r_{neighborhood}$ est la note moyenne des voisins.

**Limitations :**

- **Évolutivité :** Le filtrage collaboratif utilisateur-utilisateur rencontre des difficultés d'évolutivité en raison du temps nécessaire pour calculer les matrices de similarité pour de grandes bases d'utilisateurs.
- **Stabilité :** Les préférences des utilisateurs peuvent évoluer avec le temps, ce qui peut affecter la stabilité et la précision des recommandations.

2. **Filtrage collaboratif item-item :** Cette méthode se concentre sur la similitude entre les articles plutôt que sur les utilisateurs. Elle recommande des articles similaires à ceux que l'utilisateur a déjà aimés, ce qui permet de surmonter certaines des limitations du filtrage collaboratif utilisateur-utilisateur.

3. **Filtrage collaboratif basé sur un modèle :** Cette variante utilise des algorithmes et des modèles pour prédire les préférences des utilisateurs à partir de l'ensemble des données, en intégrant souvent des techniques d'apprentissage automatique pour améliorer la précision des recommandations.

   Les méthodes pour créer des systèmes de recommandation collaboratifs peuvent être regroupées comme suit:

   - **Algorithmes de clustering (K-Nearest Neighbors - KNN, etc)** : Ces algorithmes regroupent les utilisateurs ou les items en clusters en fonction de leur similarité. Par exemple, le KNN peut recommander des items en identifiant les utilisateurs ayant des préférences similaires et en suggérant des items qu'ils ont appréciés.
   - **Techniques de factorisation de matrice** : Ces méthodes décomposent la matrice d'interaction utilisateur-item en matrices de dimensions inférieures, capturant les facteurs latents qui expliquent les notes observées. Les techniques clés incluent :
     - **Décomposition en valeurs singulières (SVD)** : Une méthode couramment utilisée pour la réduction de la dimensionnalité, qui factorise la matrice en vecteurs et valeurs singulières.

- **Factorisation matricielle probabiliste (PMF)** : Une approche probabiliste qui modélise les données avec une distribution gaussienne, en estimant les facteurs latents à l'aide du maximum de vraisemblance.
- **Factorisation matricielle non négative (NMF)** : Cette méthode garantit que tous les facteurs sont non négatifs, ce qui peut être particulièrement utile pour l'interprétabilité dans certaines applications.

Pour former un modèle d'apprentissage automatique en utilisant cette approche, nous procédons généralement à une factorisation matricielle de la matrice de notation $r_{ui}$ pour prédire les notes des paires utilisateur-item $(u, i)$ inconnu. La formule utilisé pour prédire les notes diffère selon si on a des des retours implicites ou explicite de la part des utilisateurs.

**Retours explicites :** Une méthode courante consiste à modéliser les interactions utilisateur-item $(u, i)$ à l'aide d'un modèle linéaire. L'objectif est d'apprendre les facteurs latents $p_u$ et $q_i$ qui représentent respectivement les utilisateurs et les items. Ceci est réalisé en minimisant une fonction de perte d'erreur quadratique moyenne (MSE) régularisée sur les paires observées $(u, i)$ pour lesquelles la note $r_{ui}$ est connue (la méthode PMF).

Dans PMF, la notation $r_{ui}$ est modélisée comme le produit intérieur des facteurs latents $p_u$ et $q_i$, plus un terme de bruit. La fonction de perte intègre à la fois l'erreur quadratique moyenne (MSE) entre les notes prédites et les notes réelles, ainsi qu'un terme de régularisation pour éviter le surajustement "overfitting". Cette méthode capture efficacement la structure sous-jacente des données, permettant des prévisions précises pour les notes non observées.

$$\hat{r}_{ui} = q_i^T p_u$$
$$L = \sum_{(u,i)\in K} [(r_{ui} - \hat{r}_{ui})^2 + \lambda(\|p_i\|_2^2 + \|q_i\|_2^2)]$$

La fonction de perte peut être minimisée de deux manières principales. La première approche consiste à utiliser la descente de gradient stochastique (SGD). Bien que la SGD soit simple à implémenter, elle peut poser des problèmes en raison du fait que $p_u$ et $q_i$ sont tous deux inconnus, ce qui rend la fonction de perte non convexe. Pour surmonter ce défi, une alternative consiste à fixer $p_u$ ou $q_i$, ce qui transforme le problème en une régression linéaire convexe, pouvant être facilement résolue par les moindres carrés ordinaires (OLS). Cette méthode, connue sous le nom de moindres carrés alternés (ALS), permet une parallélisation et une accélération significatives du calcul.

L'algorithme PMF a été ensuite généralisé par l'algorithme de décomposition en valeurs singulières (SVD), qui a introduit des termes de biais dans

le modèle. Plus précisément, $b_u$ et $b_i$ représentent les écarts de notation observés de l'utilisateur $u$ et de l'item $i$, respectivement, tandis que $\mu$ est la note moyenne globale. Ces termes de biais expliquent souvent une grande partie des variations des notes observées, car certains items reçoivent systématiquement de meilleures ou de moins bonnes notes, et certains utilisateurs sont plus ou moins généreux dans leurs évaluations.

$$\hat{r}_{ui} = \mu + b_u + b_i q_i^T p_u$$

**Retours implicites [1]:** La méthode SVD peut être adaptée aux ensembles de données de retour implicite en tenant compte de ce retour comme une mesure indirecte de la confiance. Supposons que le retour implicite $t_{ui}$ représente le pourcentage du film $i$ que l'utilisateur $u$ a visionné. Par exemple, $t_{ui} = 0$ signifie que $u$ n'a jamais regardé $i$, $t_{ui} = 0.1$ signifie qu'il en a regardé seulement 10%, et $t_{ui} = 2$ signifie qu'il l'a regardé deux fois. Intuitivement, un utilisateur est plus susceptible d'être intéressé par un film qu'il a regardé deux fois que par un film qu'il n'a jamais vu. Nous définissons donc une matrice de confiance $c_{ui}$ et une matrice de notation $r_{ui}$ en conséquence.

Ces matrices permettent de modéliser de manière plus précise l'intérêt potentiel des utilisateurs pour différents items en fonction de leur comportement implicite, ce qui améliore la qualité des recommandations.

$$c_{ui} = 1 + \alpha t_{ui}$$
$$r_{ui} = \begin{cases} 1 & \text{if } t_{ui} > 0 \\ 0 & \text{if } t_{ui} = 0 \end{cases}$$

Où le paramètre $\alpha$ représentant combien fait on confiance à l'interaction implicite.

Nous pouvons ensuite modéliser le $r_{ui}$ observé en utilisant le même modèle linéaire que celui employé pour la SVD, mais avec une fonction de perte légèrement modifiée. Contrairement au cas explicite, où une interaction non observée entre l'utilisateur $u$ et l'item $i$ est représentée par $r_{ui} =$"?", dans le cas de retour implicite, si l'utilisateur $u$ n'a jamais interagi avec l'élément $i$, nous avons $r_{ui} = 0$.

Nous calculons la perte sur toutes les paires $(u, i)$, en pondérant chaque terme de perte par la confiance $c_{ui}$ que l'utilisateur $u$ accorde à l'élément $i$. Cette pondération reflète la certitude que $u$ aime $i$, ce qui permet d'adapter la fonction de perte pour mieux capturer les préférences implicites des utilisateurs.

$$L = \sum_{(u,i)\in K} [c_{ui}(r_{ui} - \hat{r}_{ui})^2 + \lambda(\|p_i\|_2^2 + \|q_i\|_2^2)]$$

**Retours explicites et implicites[2]:** L'algorithme $SVD^{++}$ est particulièrement utile lorsque nous avons accès à la fois à des retours

explicites et implicites. Cette approche est avantageuse, car les utilisateurs interagissent généralement avec de nombreux items (retours implicites) mais n'en évaluent qu'un petit sous-ensemble (retours explicites). Pour chaque utilisateur $u$, notons l'ensemble $N(u)$ des items avec lesquels $u$ a interagi. Nous supposons alors qu'une interaction implicite avec un item $j$ est associée à un vecteur latent $z_j \in R^f$.

L'algorithme $SVD^{++}$ étend le modèle linéaire de la SVD en incluant, dans la représentation de l'utilisateur, une somme pondérée de ces facteurs latents $z_j$. Cela permet de mieux capturer l'influence des interactions implicites sur les préférences des utilisateurs, améliorant ainsi la précision des recommandations.

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T(p_u + |N(u)|^{-0.5} \textstyle\sum_{j \in N(u)} z_j)$$

Un des principaux avantages des approches basées sur des modèles est leur capacité à surmonter les défis liés à la rareté "sparsity" et à l'évolutivité des données.

- **Méthodes d'apprentissage profond** : Ces méthodes utilisent des réseaux neuronaux pour modéliser des relations complexes dans les préférences des utilisateurs et les caractéristiques des items. Des techniques telles que les auto-encodeurs, le filtrage collaboratif neuronal et les réseaux neuronaux récurrents (RNN) sont employées pour capturer des relations et interactions complexes au sein des données.

La perte ponctuelle et la perte par paire sont deux types de fonctions de perte utilisées en apprentissage automatique, notamment pour les tâches de classement, comme dans les systèmes de recommandation et la recherche d'informations. Ces fonctions de perte se distinguent par leur manière de gérer le classement des éléments. Le choix entre une fonction de perte ponctuelle et par paire dépend de la tâche spécifique ainsi que de l'importance accordée à l'optimisation de l'ordre de classement par rapport aux scores des éléments individuels. Supposant $y_i$ et $y_j$ sont les scores réels, et $\hat{y}_i$ et $\hat{y}_j$ sont les scores prédits pour les éléments $i$ et $j$..

**Perte ponctuelle** Les fonctions de perte ponctuelle traitent le problème de classement comme un problème de régression ou de classification pour des éléments individuels. Chaque élément est considéré indépendamment, et le modèle prédit un score ou une pertinence pour chaque élément séparément.

- **Erreur quadratique moyenne (MSE)** : Utilisée lorsque la tâche consiste à prédire un score continu pour chaque élément.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

- **Perte d'entropie croisée** : Utilisée lorsque la tâche consiste à prédire une étiquette binaire (par exemple, pertinente ou non pertinente).

$$\text{Entropie_croisée} = -\frac{1}{N}\sum_{i=1}^{N}(y_i \times \log(\hat{y}_i) + (1 - y_i) \times \log(1 - \hat{y}_i))$$

**Perte par paires** Les fonctions de perte par paires prennent en compte l'ordre relatif entre les paires d'éléments. Le modèle prédit la préférence entre deux éléments, en cherchant à classer l'élément le plus pertinent plus haut que l'élément le moins pertinent.

- **Hinge Loss** : Utilisée dans les machines à vecteurs de support (SVM) pour le classement.

$$\text{Hinge_Loss} = \max(0, 1 - (\hat{y}_i - \hat{y}_j) \cdot (y_i - y_j))$$

- **RankNet Loss** : Une fonction de perte par paires utilisée dans RankNet.

$$\text{RankNet_Loss} = -\frac{1}{N}\sum_{i,j}(y_{ij} \times \log(\sigma(\hat{y}_i - \hat{y}_j)) + (1 - y_{ij}) \times \log(1 - \sigma(\hat{y}_i - \hat{y}_j)))$$

  où $y_{ij}$ est égale à 1 si l'élément $i$ est préféré à l'élément $j$, et 0 sinon. $\sigma$ est la fonction sigmoïde.

**NCF [3] : Les éléments et les utilisateurs sont encodés en one-hot**

Les couches suivantes sont des couches d'intégration, qui transforment les entrées éparses en vecteurs denses ou latents à partir de la couche d'entrée. Les vecteurs latents ainsi obtenus sont ensuite introduits dans une architecture neuronale multicouche, qui a pour rôle de mapper ces vecteurs latents aux scores de probabilité prédits. Ces couches sont conçues pour capturer des relations complexes entre utilisateurs et items à partir des données. La couche de sortie produit le score prédit $Y_{\text{pred}}(ui)$, représentant la probabilité que l'utilisateur $u$ interagisse avec l'élément $i$.

Pour entraîner le modèle, les auteurs ont utilisé une fonction de perte ponctuelle, visant à minimiser la différence entre la valeur cible $Y(ui)$ et la valeur prédite correspondante.

**Modèle : Fusion de la factorisation matricielle générale (GMF) et de la couche perceptron multicouche (MLP) sous le nom de NeuMF**

L'idée principale derrière cette fusion est que le GMF modélise mieux les dépendances linéaires, tandis que le MLP est plus efficace pour capturer les dépendances non linéaires dans les interactions utilisateur-item.

Dans cette approche, les vecteurs d'utilisateur et d'item pour le MLP et le GMF sont formés séparément, ce qui permet à chaque modèle d'atteindre une optimalité individuelle. Ensuite, les sorties de ces couches sont concaténées et passées à travers une couche NeuMF, qui produit les prédictions finales et permet la rétropropagation à partir de la fonction de perte.

[4] compare la factorisation matricielle à la méthode NCF. L'étude révèle qu'avec un réglage adéquat des hyperparamètres, un simple produit scalaire surpasse systématiquement les similitudes apprises proposées par les méthodes comme le MLP. L'article souligne également que, bien que les MLP soient théoriquement puissants, ils posent des problèmes de coût dans les environnements de production pour la recommandation d'articles. En revanche, les produits scalaires permettent l'application d'algorithmes de recherche très efficaces. La conclusion de l'article est que, bien que les MLP puissent être efficaces en tant que combinateurs d'intégration, les produits scalaires restent un choix par défaut préférable, en raison de leurs performances supérieures et de leur efficacité pratique dans les systèmes de recommandation réels.

**Approche Softmax des réseaux neuronaux profonds [5]**

Ce modèle estime la probabilité qu'un utilisateur $U$ regarde une vidéo $W_t$ à partir d'un corpus de vidéos $i$ dans un contexte $C$. $u$ représente l'intégration dense de l'utilisateur, et $v_j$ l'intégration d'une vidéo candidate, toutes deux dérivées des historiques de visionnage.

Les intégrations utilisateurs sont apprises en fonction de l'historique et transmises à la couche *softmax* pour classer les vidéos. La probabilité d'un visionnage est représentée par 1, sinon par 0.

Pour éviter le coût élevé du *softmax* sur de nombreux éléments, YouTube utilise l'échantillonnage négatif, qui réduit la taille de l'ensemble d'apprentissage et améliore les performances.

Une fois les embeddings générés, le réseau de notation recherche le voisin le plus proche dans l'espace produit des vecteurs utilisateur et item, générant ainsi les $N$ meilleures recommandations.

YouTube apprend les embeddings des vidéos via un vocabulaire fixe. L'historique de l'utilisateur, représenté par des identifiants vidéo épars, est mappé sur des vecteurs denses, puis concaténé avec d'autres attributs comme l'âge ou le pays. Ces caractéristiques sont traitées par des $ReLU$ multicouches, et optimisées via la descente de gradient. Les embeddings finaux sont ensuite utilisés pour la recommandation via l'algorithme du voisin le plus proche.

**Recommandation comme prédiction de séquence**

Lorsque nous interagissons avec des items, nous le faisons de manière séquentielle : nous sélectionnons un item, interagissons avec lui, puis passons au suivant. En tant qu'utilisateurs, nous créons donc une séquence d'items qui reflète l'ordre dans lequel nous y accédons. Dans ce contexte,

un système de recommandation basé sur la séquence utilise ces interactions pour prédire le prochain l'item de la séquence.

Pour effectuer cette prédiction, le système attribue à chaque item une probabilité $P$ pour un utilisateur donné, indiquant la probabilité que cet utilisateur interagisse ensuite avec l'item, en fonction de son historique d'interactions.

Le système modélise chaque utilisateur en fonction de son historique d'interactions avec les items. Plusieurs approches peuvent être utilisées pour résoudre ce problème de prédiction de séquence :

- L'utilisateur est représenté par la moyenne des vecteurs des éléments avec lesquels il a interagi.
- Les réseaux neuronaux récurrents (RNN) sont entraînés sur l'historique séquentiel des interactions de l'utilisateur pour générer une représentation (embedding) de l'utilisateur.
- Un réseau neuronal convolutionnel (CNN) peut être utilisé en divisant la séquence d'interactions en fenêtres d'apprentissage, puis en déplaçant cette fenêtre unidimensionnelle sur la séquence pour obtenir la représentation utilisateur.

Au début des années 2000, le filtrage collaboratif basé sur les K-Nearest Neighbors (KNN-CF) [6] était la méthode prédominante pour générer des recommandations. Cependant, avec l'expansion du commerce électronique et la diversification croissante des gammes de produits, le KNN-CF a rencontré des défis importants pour gérer et faire évoluer efficacement les recommandations [7]. En 2009, le paysage des systèmes de recommandation a changé de manière significative avec l'essor des techniques de factorisation matricielle, qui ont remporté le concours Netflix Prize (Netflix Prize Competition, 2009). Cette méthode est rapidement devenue dominante pour les prédictions de note [8].

Le concours Netflix, dont l'objectif était d'améliorer la précision des recommandations de films, a illustré l'efficacité des techniques de factorisation matricielle (Koren & Bell, 2015) [9]. L'adoption généralisée de ces méthodes a conduit à la mise en place de l'erreur quadratique moyenne (RMSE) comme mesure standard pour évaluer les systèmes de recommandation, et a consolidé la prédiction des notes comme une tâche centrale dans le domaine. De plus, le succès du concours Netflix a inspiré la création de Kaggle, une plateforme majeure de concours en science des données, qui est devenue un lieu clé pour faire avancer les méthodologies d'apprentissage automatique (Kaggle, 2010).

Contrairement au KNN-CF, qui repose sur des similitudes explicites entre utilisateurs ou items, la factorisation matricielle identifie des facteurs latents à partir des données existantes pour révéler des modèles et améliorer les prédictions au fil du temps [2]. Ces facteurs latents sont abstraits et non directement liés à des caractéristiques du monde réel, permettant ainsi des recommandations plus nuancées et évolutives.

Des techniques telles que la décomposition en valeurs singulières (SVD) et la factorisation matricielle sont employées pour gérer des données à grande échelle et améliorer la précision des recommandations [2]. En outre, les systèmes basés sur des modèles génèrent efficacement des recommandations en utilisant des modèles appris, même pour les utilisateurs ou items non explicitement représentés dans les données d'apprentissage [10]. Par exemple, le filtrage collaboratif qui se concentre sur la similarité entre les items plutôt que sur les utilisateurs, est particulièrement efficace pour atténuer les problèmes de matices creuses et améliorer la précision des recommandations [11].

**Inconvénient:** Les systèmes de recommandation basés sur le filtrage collaboratif rencontrent des défis liés au problème du démarrage à froid et aux préoccupations en matière de confidentialité, en raison de la nécessité de partager les données des utilisateurs.

## 2.3 Recommandations basée sur le contenu

Le filtrage basé sur le contenu se concentre sur l'analyse des caractéristiques des items et du profil de l'utilisateur pour identifier ce qu'ils pourraient apprécier. Le système apprend d'abord les préférences de l'utilisateur, puis recommande des éléments similaires en posant la question : "Qu'est-ce que cet utilisateur pourrait aimer ?" Contrairement à d'autres approches, il n'est pas nécessaire de disposer d'un grand nombre d'utilisateurs pour faire des recommandations efficaces. Cependant, il est essentiel de suivre les descriptions des items et de maintenir un profil utilisateur qui peut être utilisé pour faire correspondre les items appropriés.

La capacité à traiter et à extraire automatiquement ces descriptions relève du traitement du langage naturel (titre, description, etc), voire de la vision par ordinateur (image), selon les types des caractéristiques en jeu.

**Stratégie 1:** la stratégie prend en compte non seulement la pertinence des items, mais aussi leur correspondance (les caractéristiques des aimées) avec les préférences individuelles de l'utilisateur.

Les prédictions des scores des éléments sont effectuées en utilisant des poids normalisés ($1/\sqrt{\sum_{\forall i} \text{caractéristique}_i = True}$) afin d'éviter de surévaluer les items qui couvrent un grand nombre de caractéristiques. Cette normalisation garantit que les items ne reçoivent pas un avantage disproportionné simplement parce qu'ils incluent plus de caractéristiques, permettant ainsi une évaluation plus équilibrée et précise des items en fonction des préférences de l'utilisateur.

**Stratégie 2 (TF-IDF):** Cette stratégie prend en compte non seulement la pertinence d'une caractéristique (par exemple, si un item $x$ couvre de nombreuses caractéristiques), mais aussi l'importance relative de ces caractéristiques. L'importance est déterminée en fonction de leur rareté ou fréquence, ainsi que de leur contribution à la similarité ou à la diversité des éléments. Pour ce faire, chaque valeur associée à une caractéristique est divisée par le nombre d'autres items où cette même caractéristique apparaît, ce qui est typique dans les cas où la similarité est un facteur clé.

Le score final pour la correspondance entre l'item et l'utilisateur sera calculé comme suit :

$$\text{Item_caractéristiques} \times IDF \times \text{User_Taste}$$

Pour calculer l'IDF de chaque terme, on divise chaque valeur de la caractéristique par le nombre d'autres items dans lesquels cette même caractéristique apparaît. Cette pondération permet de donner plus de poids aux caractéristiques rares et distinctives, améliorant ainsi la qualité des recommandations basées sur les préférences de l'utilisateur.

**Avantage:** Les techniques basées sur le contenu peuvent surmonter le problème du démarrage à froid "cold start", à condition que les nouveaux éléments disposent d'une description suffisante. Elles sont particulièrement efficaces dans les domaines caractérisés par un contenu dynamique. Leur capacité à s'adapter aux évolutions des préférences des utilisateurs et aux changements dans le paysage du contenu rend cette approche très efficace.

**Inconvénient:** Cette méthode nécessite une connaissance approfondie des caractéristiques des items pour fournir des recommandations précises. Les informations nécessaires peuvent ne pas être disponibles pour tous les items.

## 2.4 Approche hybride

LightFM [12]: Les méthodes de filtrage collaboratif basées sur la factorisation matricielle, bien qu'efficaces dans de nombreux contextes, rencontrent des difficultés dans les scénarios de démarrage à froid, où les données d'interaction pour les nouveaux items et utilisateurs sont insuffisantes. Pour pallier ce problème, les approches hybrides combinent les données de métadonnées connues des items ou des utilisateurs avec les techniques de factorisation matricielle. La bibliothèque Python LightFM est un exemple populaire d'algorithme hybride.

Dans LightFM, nous supposons que chaque utilisateur $u$ a un ensemble d'annotations de balises $A^u(u)$ — par exemple, "homme", "âge $< 30$ ans", etc. — et chaque item $i$ a un ensemble d'annotations $A^i(i)$ — par exemple, "prix $> 100\$$", "livre", etc. Nous modélisons chaque balise utilisateur avec un facteur latent $x_a^U \in R^f$ et un terme de biais $b_a^U \in R$. La représentation vectorielle de l'utilisateur $p_u$ et son biais associé $b_u$ sont exprimés comme la somme de ces termes $x_a^U$ et $b_a^U$, respectivement. Nous appliquons une approche similaire pour les balises d'items, utilisant des facteurs latents $x_a^I \in R^f$ et des termes de biais $p_u, q_i, b_u, b_i$. Une fois ces termes définis, nous utilisons le même modèle linéaire que celui utilisé en SVD pour décrire la relation entre ces termes et $r_{ui}$.

Cas d'utilisation de LightFM :

Démarrage à froid : Si nous avons un nouvel item $i$ avec des balises connues $A^I(i)$, nous pouvons utiliser les vecteurs latents $x_a^I$ (obtenus en ajustant notre modèle sur les données antérieures) pour calculer l'intégration $q_i$ de l'item et ainsi estimer la note $r_{ui}$ pour tout utilisateur $u$.

Absence de balises disponibles : Si aucune métadonnée n'est disponible pour les items ou les utilisateurs, nous utilisons des annotations indicatrices, c'est-à-

dire des balises distinctes pour chaque utilisateur et item. Dans ce cas, les matrices de caractéristiques des utilisateurs et des items sont des matrices d'identité, et LightFM se réduit à une méthode de filtrage collaboratif classique telle que la SVD.

Basé sur le contenu ou hybride : Si nous n'utilisons que des balises utilisateur ou item sans annotations indicatrices, LightFM se comporte comme un modèle basé sur le contenu. En pratique, pour tirer parti des interactions utilisateur-item, nous combinons les balises connues avec des annotations indicatrices distinctes pour chaque utilisateur et item.

# 3 Défis des systèmes de recommandation

***Problème de démarrage à froid:*** Ce défi se pose lorsque de nouveaux utilisateurs intègrent le système ou que de nouveaux items sont ajoutés à la base de données, entraînant un manque d'information pour effectuer des recommandations pertinentes.

***Problème d'attaque de shilling :*** Ce problème survient lorsqu'un utilisateur malveillant cherche à manipuler la popularité d'un item en biaisant les évaluations sur des items cibles, en vue de les augmenter ou de les diminuer.

***Problème de synonymie:*** Ce défi apparaît lorsque des éléments similaires ou liés ont des noms ou des entrées différents.

***Problème de latence:*** Spécifique aux méthodes de filtrage collaboratif, ce problème se manifeste lorsque de nouveaux items sont fréquemment ajoutés à la base de données, mais que le système est incapable de les recommander efficacement.

***Problème de parcimonie (Sparsity):*** Ce problème se produit lorsque certaines données attendues manquent dans l'ensemble de données.

***Problème du mouton gris:*** Ce défi est caractéristique des approches purement collaboratives, où les préférences d'un utilisateur ne correspondent à aucun groupe d'utilisateurs similaires.

# 4 Considérations Qualitatives dans les Systèmes de Recommandation

Dans les systèmes de recommandation, certains concepts sont essentiels pour créer des recommandations équilibrées et centrées sur l'utilisateur. Voici une brève explication de chacun:

***Diversité :*** Elle garantit que les items recommandés ne soient pas trop similaires entre eux, couvrant un large éventail de genres, catégories ou sujets.

Les mesures mathématiques utilisées pour déterminer la similarité entre deux vecteurs:

- Similarité cosinus : Mesure l'angle cosinus entre les vecteurs.
- Produit scalaire : Prend en compte à la fois l'angle cosinus et la magnitude des vecteurs.
- Distance euclidienne : Mesure la distance au carré entre les éléments de deux vecteurs.
- Similarité Pearson : Coefficient mesurant la corrélation linéaire entre deux vecteurs.

Une deuxième stratégie consiste à chercher les éléments des vecteurs qui sont le plus aimé par un certain utilisateurs. Le Notamment, le *test du chi carré* $X^2$ [13] est un test statistique utilisé pour déterminer s'il existe une association significative entre deux variables catégorielles. Dans le contexte du filtrage basé sur le contenu, il peut être employé pour identifier les caractéristiques ou attributs des items qui sont fortement associés aux préférences des utilisateurs.

***Sérendipité :*** La sérendipité dans les recommandations est la découverte agréable d'éléments inattendus mais intéressants et pertinents. Elle présente aux utilisateurs du contenu qu'ils n'auraient peut-être pas découvert par eux-mêmes.

***Équité :*** L'équité dans les systèmes de recommandation garantit que les recommandations sont justes et impartiales, représentant un large éventail d'items de manière équitable sans favoriser de manière disproportionnée certains items, genres ou groupes d'utilisateurs.

***Biais :*** Il existe quatre types potentiels de biais dans les systèmes de recommandation :

- Biais de l'ensemble de données : Par exemple, un ensemble de données peut afficher des notes systématiquement plus élevées par rapport à un autre, ce qui peut être dû à des facteurs comme les différences culturelles dans les lieux où les données ont été collectées.
- Biais de l'utilisateur : Ces biais peuvent provenir de caractéristiques spécifiques à l'utilisateur, comme le niveau d'éducation, le statut socio-économique, etc.
- Biais de l'item : Peut survenir lorsqu'un système favorise certains items ou groupes d'items au détriment d'autres.
- Biais temporel, ou lié au temps: Le temps peut influencer les notes des films et affecte également la manière dont les utilisateurs évaluent ces films pour diverses raisons.

# 5 Évaluation

Les mesures d'évaluation offrent une approche globale pour juger des performances des systèmes de recommandation, en tenant compte à la fois des perspectives centrées sur l'utilisateur et sur l'efficacité de l'algorithme.

**Mesures de précision:** La précision des recommandations est souvent évaluée à travers des tâches telles que la prédiction des notes. Cette tâche mesure la capacité de l'algorithme à prédire les notes en les comparant aux notes réelles dans un ensemble de test.

Dans le domaine des systèmes de recommandation, les mesures de précision sont essentielles pour évaluer la performance des algorithmes. Parmi les mesures les plus couramment utilisées figurent l'erreur absolue moyenne (MAE) et l'erreur quadratique moyenne (RMSE). La MAE mesure la distance moyenne entre les notes prédites et les notes réelles, tandis que la RMSE, en élevant les erreurs au carré avant de les moyenner, accorde plus de poids aux erreurs importantes.

**Mesures de classement:**

Les mesures de classement évaluent l'efficacité d'un algorithme à organiser les recommandations pour des tâches spécifiques, en se concentrant sur la manière dont les articles sont hiérarchisés pour les utilisateurs.

Dans les systèmes de recommandation, plusieurs mesures importantes sont utilisées pour juger de la qualité des listes proposées :

*Utilité de la demi-vie :*

- Priorise les éléments les mieux classés en haut de la liste.
- Considère uniquement les notes positives comme pertinentes.
- Prend en compte des scores d'évaluation non binaires.

*Gain cumulé actualisé (DCG) :*

- Évalue la qualité du classement en mettant l'accent sur les articles en début de liste.
- Le DCG présente des limites dues à la variation du nombre de recommandations par utilisateur. Il est normalisé par le gain cumulé actualisé normalisé (NDCG), ce qui permet une comparaison plus uniforme entre les utilisateurs.

*Rang réciproque moyen (MRR) :*

- Calcule la moyenne des rangs réciproques de la première recommandation pertinente pour chaque utilisateur.
- Met l'accent sur l'item le mieux classé et est particulièrement utile lorsque l'ordre des recommandations est crucial. Les scores de MRR varient entre 0 et 1, avec des valeurs plus élevées indiquant une meilleure performance.

*Précision moyenne (MAP) :*

- Calcule la précision moyenne pour chaque utilisateur, puis la moyenne sur tous les utilisateurs.

- Prend en compte à la fois l'ordre et la pertinence des items recommandés, offrant une mesure complète de l'efficacité des classements.

Ces mesures permettent d'évaluer la qualité des recommandations en considérant différents aspects des performances des systèmes de recommandation.

**Mesures de l'impact des recommandations:** Les mesures d'impact servent à évaluer diverses qualités importantes pour les utilisateurs en fonction des objectifs de l'algorithme. Par exemple, si l'algorithme vise à améliorer la diversité des produits par utilisateur, les mesures doivent évaluer la variété des produits recommandés. D'autre part, si l'algorithme cible des produits de niche, l'évaluation doit se concentrer sur le fait de savoir si les résultats recommandés incluent un pourcentage plus élevé de produits non populaires par rapport aux autres algorithmes.

Mesurer l'impact des recommandations dans les systèmes de recommandation nécessite de prendre en compte divers facteurs et compromis. Un des défis majeurs est la gestion des utilisateurs sans données d'interaction préalable, rendant difficile la prédiction de leurs préférences. Trouver un équilibre entre la précision des recommandations et la couverture des utilisateurs devient alors crucial, car l'incertitude demeure quant au nombre exact d'utilisateurs que le système doit servir.

Mesurer l'impact des recommandations implique de jongler entre plusieurs compromis, notamment entre la précision, la couverture des utilisateurs, la couverture des articles, la diversité, la nouveauté et la sérendipité. L'utilisation de mesures comme l'indice de Gini et l'entropie de Shannon permet de quantifier ces différents aspects, fournissant ainsi des informations précieuses sur l'efficacité du système de recommandation et sur l'expérience utilisateur qu'il offre.

**Quantifier la valeur commerciale des recommandations:** Les métriques standards ne capturent souvent pas entièrement l'impact réel des recommandations sur les clients ni leur alignement avec les stratégies commerciales. Pour remédier à cela, les tests A/B sont essentiels, bien qu'ils nécessitent plus de ressources et de temps. Les indicateurs clés mesurés par les tests A/B incluent :

- Taux de Clics (CTR) : Calculé en divisant le nombre de clics reçus par un article par le nombre total de fois où il a été recommandé. Un CTR élevé est généralement corrélé à une probabilité accrue d'achat.

- Métriques d'Adoption et de Conversion : Comme le "Long CTR" de YouTube et le "Take Rate" de Netflix, qui évaluent les interactions des utilisateurs, telles que le pourcentage de vidéos visionnées ou la fréquence de visionnage des contenus recommandés.

- Métriques Spécifiques au Domaine : Pour des plateformes comme LinkedIn, des mesures telles que le nombre de contacts établis après une recomman-

dation d'offre d'emploi sont cruciales pour évaluer l'efficacité des recommandations.

Bien que le CTR et les taux d'adoption montrent le succès des recommandations en termes de vues ou d'achats, l'impact sur les ventes et le chiffre d'affaires reste difficile à quantifier. L'analyse des changements dans la répartition des ventes après l'implémentation des recommandations fournit des informations directes, mais nécessite une compréhension approfondie des évolutions dans les modèles de vente. Évaluer le comportement et l'engagement des utilisateurs est également essentiel, bien que plus complexe dans des contextes où les taux de désabonnement sont faibles.

# References


[1] Y. Hu, Y. Koren, C. Volinsky, Collaborative filtering for implicit feedback datasets, in: 2008 Eighth IEEE international conference on data mining, Ieee, 2008, pp. 263–272.

[2] Y. Koren, Factorization meets the neighborhood: a multifaceted collaborative filtering model, in: Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining, 2008, pp. 426–434.

[3] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, T.-S. Chua, Neural collaborative filtering, in: Proceedings of the 26th international conference on world wide web, 2017, pp. 173–182.

[4] S. Rendle, W. Krichene, L. Zhang, J. Anderson, Neural collaborative filtering vs. matrix factorization revisited, in: Proceedings of the 14th ACM Conference on Recommender Systems, 2020, pp. 240–248.

[5] P. Covington, J. Adams, E. Sargin, Deep neural networks for youtube recommendations, in: Proceedings of the 10th ACM conference on recommender systems, 2016, pp. 191–198.

[6] A. Strehl, J. Ghosh, R. Mooney, Impact of similarity measures on web-page clustering, in: Workshop on artificial intelligence for web search (AAAI 2000), Vol. 58, 2000, p. 64.

[7] G. Adomavicius, A. Tuzhilin, Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions, IEEE transactions on knowledge and data engineering 17 (6) (2005) 734–749.

[8] Y. Koren, R. Bell, C. Volinsky, Matrix factorization techniques for recommender systems, Computer 42 (8) (2009) 30–37.

[9] Y. Koren, S. Rendle, R. Bell, Advances in collaborative filtering, Recommender systems handbook (2021) 91–142.

[10] R. M. Bell, Y. Koren, Lessons from the netflix prize challenge, Acm Sigkdd Explorations Newsletter 9 (2) (2007) 75–79.

[11] B. Sarwar, G. Karypis, J. Konstan, J. Riedl, Item-based collaborative filtering recommendation algorithms, in: Proceedings of the 10th international conference on World Wide Web, 2001, pp. 285–295.

[12] M. Kula, Metadata embeddings for user and item cold-start recommendations, arXiv preprint arXiv:1507.08439 (2015).

[13] D. Wang, Y. Liang, D. Xu, X. Feng, R. Guan, A content-based recommender system for computer science publications, Knowledge-based systems 157 (2018) 1–9.